# DOCUMENTO DE GESTIÓN ÁGIL Y ESPECIFICACIÓN TÉCNICA SCRUM

## Proyecto: S&J-GEEK - Plataforma Web Backend & Frontend

### INFORMACIÓN GENERAL

| Campo | Detalle |
|---|---|
| **Proyecto** | S&J-GEEK - Sistema de Gestión, Catálogo y Ranking Geek (Node.js/Express + MySQL + Vanilla JS + CSS3 + HTML5) |
| **Estudiante(s)** | Stefani Sánchez – Jakelin Quino |
| **Docente(s)** | Eduin Salas - André López |
| **Ruta** | Express |
| **Periodo / Fechas** | 26/10/2026 – 31/10/2026 |
| **Repositorio Backend** | https://github.com/JakelinMishelQuinoMartinez/sj-geek-backend |
| **Repositorio Frontend** | https://github.com/JakelinMishelQuinoMartinez/sj-geek-frontend |
| **Tablero de Seguimiento** | https://jakelinquino303s-team.monday.com/workspaces/18190428 |

---

### Estructura Completa del Documento

- **01** | Situación Problema, Alcance y Arquitectura Técnica
- **02** | Épicas y levantamiento Detallado de Requerimientos (RF-01 a RF-12 & RNF-01 a RNF-08)
- **03** | Roles Asignados y Estrategia de Control de Versiones (Git Branching & Commits)
- **04** | Definición de Sprints y Cronograma Diario Detallado (26/10 al 30/10)
- **05** | Historias de Usuario Exhaustivas con Sintaxis Gherkin (HU-01 a HU-07)
- **06** | Algoritmo Ponderado Bayesiano, Herramienta de Seguimiento y Evidencias
- **07** | Matriz de Verificación de Casos de Pruebas y Cierre

---

# 01 | Situación Problema, Alcance y Arquitectura Técnica

## 1.1 Situación Problema

En la actualidad, las plataformas de recomendación y entretenimiento suelen estar dispersas o presentar arquitecturas monolíticas rígidas que dificultan el filtrado interactivo de contenido geek (películas, animes y series). Además, existe una falta de consistencia de datos en los sistemas de calificación tradicionales cuando las interacciones de los usuarios (como reseñas, ratings e interacciones de likes/dislikes) se ejecutan de manera concurrente sin control transaccional estricto en bases de datos relacionales.

### 1.1.1 Detalles de Arquitectura Técnica Backend & Frontend

- **Capas del Backend (Node.js + Express):**
  - **/config:** Variables de entorno (dotenv) y pool de conexiones MySQL2 con promesas.
  - **/routes:** Definición de endpoints RESTful modularizados por entidad con prefijo /api/v1/.
  - **/middlewares:** Autenticación (JWT Passport), autorización por rol, rate-limiting (express-rate-limit) y sanitización con express-validator.
  - **/controllers:** Manejo de peticiones HTTP, parseo de inputs, invocación de servicios y respuestas JSON estructuradas.
  - **/services:** Lógica de negocio avanzada, transacciones SQL manuales (connection.beginTransaction(), commit(), rollback()) y cálculos matemáticos (ranking bayesiano).
  - **/models:** Consultas SQL parametrizadas para evitar Inyección SQL (Prepared Statements).
- **Arquitectura del Frontend (JS Vanilla MPA):**
  - **Cliente Desacoplado:** Modular JavaScript (ES6 Modules), manipuladores de DOM asíncronos mediante fetch API y async/await.
  - **Gestión de Estado:** Token JWT almacenado en Cookies (HttpOnly / Cookies), manejo dinámico de vistas e interceptores de peticiones HTTP para manejar expiración de tokens.

## 1.2 Solución Propuesta (S&J-GEEK)

S&J-GEEK es una aplicación Web Full-Stack desacoplada diseñada para permitir a la comunidad registrar, calificar, votar y rankear contenido audiovisual geek.

- **Backend:** API RESTful modular construida en Node.js con Express, con persistencia relacional en MySQL mediante el driver/ORM compatible. Implementa autenticación segura vía JWT (`passport-jwt`, `bcrypt`), limitación de peticiones con

`express-rate-limit`, validación de esquemas con `express-validator`, documentación viva con Swagger, y manejo de transacciones ACID en MySQL (`START TRANSACTION`, `COMMIT`, `ROLLBACK`) para operaciones críticas (creación de reseñas con rating inicial y gestión de likes/dislikes).

- **Frontend:** Cliente independiente MPA desarrollado en HTML5, CSS3 y JavaScript Vanilla (JS puro), con diseño adaptativo (Responsive), consumo eficiente de la API backend y manejo limpio de errores del servidor.

---

# 02 | Levantamiento de Requerimientos

## 2.1 Definición de Épicas del Proyecto

**Épica 1: Autenticación, Usuarios y Control de Acceso (RBAC)**

- **Descripción:** Agrupa el registro de nuevos usuarios, inicio de sesión seguro mediante JWT (con bcrypt y express-rate-limit), asignación de roles (USER/ADMIN) y protección de endpoints administrativos.
- **Historias de Usuario asociadas:** HU-01, HU-02.
- **Requerimientos Funcionales asociados:** RF-01, RF-02, RF-03.

**Épica 2: Gestión del Catálogo Audiovisual y Aprobación de Contenidos**

- **Descripción:** Engloba la propuesta, registro, edición, eliminación y validación de contenidos (películas, series, animes) sin títulos duplicados, incluyendo el flujo de moderación y aprobación por parte del administrador para hacerlos visibles públicamente.
- **Historias de Usuario asociadas:** HU-04, HU-05.
- **Requerimientos Funcionales asociados:** RF-04, RF-05, RF-06, RF-07, RF-08, RF-09.

**Épica 3: Gestión de Categorías y Clasificación**

- **Descripción:** Comprende la creación, edición, listado y eliminación de categorías (Sci-Fi, Anime, Fantasía, etc.) por parte de usuarios administradores para mantener la organización y clasificación del catálogo geek.
- **Historias de Usuario asociadas:** HU-03.
- **Requerimientos Funcionales asociados:** RF-10, RF-11, RF-12, RF-13.

**Épica 4: Sistema Transaccional de Reseñas, Calificaciones e Interacciones (Likes/Dislikes)**

- **Descripción:** Permite a los usuarios publicar, editar y eliminar reseñas con calificación de 1 a 10 mediante transacciones ACID SQL en MySQL, así como reaccionar (Like/Dislike) en reseñas de otros miembros garantizando la integridad de datos.
- **Historias de Usuario asociadas:** HU-06, HU-07.
- **Requerimientos Funcionales asociados:** RF-14, RF-15, RF-16, RF-17.

**Épica 5: Algoritmo de Ranking Ponderado Bayesiano y Filtrado Avanzado**

- **Descripción:** Incluye la lógica del algoritmo bayesiano para ordenar equitativamente el catálogo geek según votos, promedios e interacciones, además de los servicios de filtrado multiparámetro por categoría y búsqueda libre.
- **Historias de Usuario asociadas:** HU-08.
- **Requerimientos Funcionales asociados:** RF-18, RF-19, RF-20.

**Épica 6: Interfaz de Usuario Frontend Desacoplada (JS Vanilla MPA)**

- **Descripción:** Cubre el diseño y desarrollo del cliente frontend independiente (Multi-Page Application) responsivo en HTML5, CSS3 y Vanilla JS con Fetch API para interactuar de forma dinámica con la API Backend RESTful.
- **Historias de Usuario asociadas:** HU-09.
- **Requerimientos Funcionales asociados:** RNF-07, RNF-08.

## 2.2 Requerimientos Funcionales (RF)

| Código | Módulo | Descripción Técnica Detallada y Criterios de Aceptación |
|---|---|---|
| **RF-01** | Autenticación | El sistema debe permitir el registro de nuevos usuarios mediante email único, nombre completo y contraseña cifrada. |
| **RF-02** | Autenticación | El sistema debe autenticar credenciales de usuario y generar un token JWT firmado. |
| **RF-03** | Permisos | El sistema debe restringir el acceso a endpoints de administración según el rol asignado en el token. |
| **RF-04** | Catálogo | El sistema debe permitir registrar una nueva oferta audiovisual con título, sinopsis, categoría, tipo, año y URL de imagen. |
| **RF-05** | Catálogo | El sistema debe permitir listar las ofertas audiovisuales disponibles en el catálogo. |
| **RF-06** | Catálogo | El sistema debe permitir editar la información de una oferta audiovisual existente. |
| **RF-07** | Catálogo | El sistema debe permitir eliminar una oferta audiovisual del catálogo. |
| **RF-08** | Catálogo | El sistema debe validar que no existan títulos audiovisuales duplicados en la base de datos. |

| Código | Módulo | Descripción Técnica Detallada y Criterios de Aceptación |
|---|---|---|
| **RF-09** | Aprobación | El sistema debe permitir cambiar el estado de aprobación de un contenido registrado a aprobado para hacerlo visible en el catálogo público. |
| **RF-10** | Categorías | El sistema debe permitir crear una nueva categoría con nombre y descripción. |
| **RF-11** | Categorías | El sistema debe permitir listar las categorías existentes. |
| **RF-12** | Categorías | El sistema debe permitir editar una categoría existente. |
| **RF-13** | Categorías | El sistema debe permitir eliminar una categoría que no tenga contenidos vinculados. |
| **RF-14** | Reseñas | El sistema debe permitir a un usuario autenticado publicar una reseña con comentario y calificación para un contenido. |
| **RF-15** | Reseñas | El sistema debe permitir a un usuario editar su propia reseña publicada. |
| **RF-16** | Reseñas | El sistema debe permitir a un usuario eliminar su propia reseña publicada. |
| **RF-17** | Reacciones | El sistema debe permitir registrar reacciones de Like o Dislike en una reseña. |
| **RF-18** | Ranking | El sistema debe permitir calcular el Ranking Ponderado Bayesiano de contenidos. |
| **RF-19** | Listados | El sistema debe permitir filtrar el catálogo por categoría y texto libre. |
| **RF-20** | Detalle | El sistema debe mostrar el detalle completo de un contenido audiovisual con sus reseñas e interacciones. |

## 2.3 Requerimientos No Funcionales (RNF)

| Código | Categoría | Especificación Técnica Exhaustiva |
|---|---|---|
| **RNF-01** | Arquitectura Capas | Desarrollo Node.js + Express con separación estricta en capas (/routes, /controllers, /services, /models, |

| Código | Categoría | Especificación Técnica Exhaustiva |
|---|---|---|
| | | /middlewares, /config, /utils). Cero lógica de base de datos en controladores. |
| **RNF-02** | Base de Datos Relacional | Uso de MySQL en 3ª Forma Normal (3FN), claves foráneas explícitas (ON DELETE RESTRICT / CASCADE según entidad), e índices b-tree en columnas frecuentemente consultadas (titulo, categoria_id, usuario_id). |
| **RNF-03** | Manejo Transaccional ACID | Implementación de transacciones SQL explícitas (connection.beginTransaction(), commit(), rollback()) en operaciones compuestas para garantizar la consistencia atómica de datos. |
| **RNF-04** | Seguridad Avanzada | Variables de entorno gestionadas con dotenv, encriptación bcrypt (salt=10), firmado JWT con secreto seguro, headers de seguridad mediante Helmet.js y rate limiting de 100 req/15min por IP. |
| **RNF-05** | Validación Sanitizada | Middleware global de validación empleando express-validator. Rechazo automático de payloads maliciosos, inyección de scripts XSS o datos fuera de rango numérico/longitud. |
| **RNF-06** | Documentación OpenAPI | Exposición de especificaciones OpenAPI 3.0 mediante swagger-ui-express en /api-docs. Todos los esquemas de petición y respuesta codificados según versionado SemVer (/api/v1). |
| **RNF-07** | Frontend Desacoplado | Interfaz SPA/MPA desarrollada 100% en HTML5 semántico, CSS3 (CSS Grid/Flexbox) y JS Vanilla modular. Prohibido el uso de frameworks pesados (React, Angular, Vue) para garantizar máxima ligereza. |
| **RNF-08** | Integración & CORS | Configuración estricta del middleware cors en Express restringiendo los orígenes permitidos (Access-Control-Allow-Origin), headers autorizados y métodos HTTP (GET, POST, PUT, DELETE). |

---

## 03 | Roles Asignados

| Rol SCRUM | Integrante(s) | Responsabilidades Clave |
|---|---|---|
| **Product Owner** | Jakelin Quino | Define y prioriza el Product Backlog, especifica los criterios de aceptación de las historias de usuario y valida los incrementos funcionales. |
| **Scrum Master** | Stefani Sánchez | Facilita las ceremonias ágiles, remueve bloqueos técnicos y operativos, y vela por el cumplimiento del flujo de trabajo. |
| **Development Team** | Stefani Sánchez & Jakelin Quino | Diseña la arquitectura backend, programa los endpoints, configura la base de datos MySQL con transacciones SQL y desarrolla la interfaz de usuario en JS puro. |
| **Revisor / Docente** | Eduin Salas & André López | Evalúa la entrega final, la consistencia de la arquitectura, el video demostrativo y el cumplimiento de los RNF. |

### 3.1 Estrategia de Control de Versiones (Git Branching & Conventional Commits)

- **Estrategia de Ramas (GitFlow Simplificado):**
  - **main:** Código de producción estable y probado.
  - **develop:** Rama de integración continua para el trabajo diario del Sprint.
  - **feature/hu-XX-nombre:** Ramas de característica creadas a partir de develop para cada Historia de Usuario.
  - **fix/nombre-bug:** Ramas para correcciones de errores detectados en la integración.
- **Estándar de Mensajes de Commit (Conventional Commits):**
  - **feat(auth):** add JWT token generation and bcrypt encryption (HU-01)
  - **feat(reviews):** implement SQL transaction for review submission (HU-04)
  - **fix(ranking):** correct Bayesian weighted formula parameters (HU-06)
  - **docs(swagger):** annotate categories and catalog endpoints

---

# 04 | Definición de Sprints y Cronograma

## Métricas del Proyecto

- **Duración del Sprint:** 1 Semana
- **Número de Sprints:** 2 Sprints
- **Fechas:** 26/10/2026 – 31/10/2026 (6 días de ejecución)

## Plan de Sprints

- **Sprint 1 (26/10/2026 - 28/10/2026): Core Backend & Autenticación**
  Configuración de servidor Express, conexión MySQL, esquemas relacionales/migraciones, auth con JWT, CRUD de Categorías y CRUD básico de Películas.
- **Sprint 2 (29/10/2026 - 31/10/2026): Transacciones, Frontend & Integración**
  Sistema de reseñas con transacciones SQL en MySQL, algoritmo de ranking ponderado, UI en JS puro (Login, Catálogo, Detalle, Admin) y documentación Swagger.

## Cronograma Diario de Trabajos (26/10/2026 – 31/10/2026)

| Fecha | Jakelin Quino (PO / Lead Dev) | Stefani Sánchez (SM / Full-Stack Dev) |
|---|---|---|
| **Lunes 26/10** | Setup de servidor Express, conexión MySQL, variables de entorno (.env) y middleware de errores. | Diseño del esquema de DB MySQL (tablas usuarios, contenidos, categorías) y scripts de migración DDL. |
| **Martes 27/10** | Desarrollo de HU-01 (Registro, Login, bcrypt, JWT) y middlewares de autenticación/RBAC (HU-02). | Desarrollo de HU-02 (CRUD Categorías) y CRUD base de Películas/Series (HU-03). |
| **Miércoles 28/10** | Documentación inicial en Swagger UI para endpoints de Auth, Categorías y Películas. Cierre de Sprint 1. | Implementación del flujo de aprobación Admin para películas (HU-03) y validación express-validator. |
| **Jueves 29/10** | Desarrollo de HU-04 (Reseñas con transacciones ACID SQL) y HU-05 (Likes/Dislikes con bloqueos). | Implementación del algoritmo bayesiano de Ranking Ponderado (HU-06) en capa de servicios MySQL. |
| **Viernes 30/10** | Desarrollo de UI Frontend en Vanilla JS (HU-07: Catálogo, Login, Detalle) e integración con API Backend. | Pruebas de integración E2E, ajustes de CORS/rate-limiting, video demostrativo y cierre final del proyecto. |
| **Sábado 31/10** | Tiempo extra por retraso. | En caso de no cumplir con los tiempos de entrega. |

### Definición de Hecho (DoD)

1. Código desarrollado, modularizado y sin advertencias en consola.
2. Validaciones con `express-validator` probadas y funcionando.
3. Transacciones SQL en MySQL validadas (`Commit` & `Rollback` exitosos).
4. Endpoint documentado e integrado en Swagger UI.
5. Frontend integrado y consumiendo la API de forma responsive.
6. Cambios integrados al repositorio con Conventional Commits.

---

## 05 | Historias de Usuario (Product Backlog)

### Backlog Priorizado

| ID | Épica | Historia de Usuario | Prioridad | Puntos | Sprint | Estado |
|---|---|---|---|---|---|---|
| **HU-01** | Épica 1: Autenticación, Usuarios y RBAC | Como usuario quiero registrarme en la plataforma con mi email, nombre y contraseña para acceder a los servicios de la plataforma. | Alta | 3... | Spr... | Por... |
| **HU-02** | Épica 1: Autenticación, Usuarios y RBAC | Como usuario quiero iniciar sesión con mis credenciales para obtener un token de autenticación. | Alta | 3... | Spr... | Por... |
| **HU-03** | Épica 3: Gestión de Categorías | Como administrador quiero crear y gestionar categorías de contenido para clasificar la oferta audiovisual. | M... | 2... | Spr... | Por... |
| **HU-04** | Épica 2: Gestión del Catálogo y Aprobación | Como usuario quiero registrar una nueva película o serie para proponer al catálogo. | Alta | 5... | Spr... | Por... |
| **HU-05** | Épica 2: Gestión del Catálogo y Aprobación | Como administrador quiero aprobar contenidos registrados por los usuarios | Alta | 2... | Spr... | Por... |

| ID | Épica | Historia de Usuario | Prioridad | Puntos | Sprint | Estado |
|---|---|---|---|---|---|---|
| | | para hacerlos visibles en el catálogo público. | | | | |
| **HU-06** | Épica 4: Reseñas, Calificaciones e Interacciones | Como usuario autenticado quiero publicar una reseña y calificación en un contenido para compartir mi opinión. | Alta | 8... | Spr... | Por... |
| **HU-07** | Épica 4: Reseñas, Calificaciones e Interacciones | Como usuario autenticado quiero reaccionar con Like o Dislike en la reseña de otro usuario para expresar mi acuerdo o desacuerdo. | M... | 5... | Spr... | Por... |
| **HU-08** | Épica 5: Ranking Ponderado Bayesiano y Filtrado | Como usuario quiero visualizar el ranking ponderado de contenidos para descubrir el material mejor valorado. | Alta | 5... | Spr... | Por... |
| **HU-09** | Épica 6: Interfaz Frontend Desacoplada | Como usuario quiero interactuar con la interfaz gráfica dinámica para explorar y filtrar el catálogo sin recargar la página. | Alta | 8... | Spr... | Por... |

## Fichas Detalladas

**FICHA HU-01: Como usuario quiero registrarme en la plataforma con mi email, nombre y contraseña para acceder a los servicios de la plataforma.**

- **Prioridad / RF:** Alta | RF-01, RF-02, RF-03, RNF-04
- **Actor:** Usuario General / Administrador
- **Criterios de Aceptación:**
  a. Email con sintaxis válida no duplicado en MySQL.
  b. Contraseña encriptada con `bcrypt`.
  c. Retorno de token JWT válido.
  d. Limitación de intentos con `express-rate-limit`.
- **Restricciones:** Expiración de JWT según `.env`.
- **Especificación BDD / Gherkin (HU-01):**

    a. **Escenario 01 (Registro Exitoso):** Dado que un nuevo visitante no registrado envía un email válido y contraseña de más de 8 caracteres, Cuando consume POST /api/v1/auth/register, Entonces el sistema almacena el usuario con la contraseña hasheada en bcrypt y responde con un código 201 Created.
    b. **Escenario 02 (Login Fallido):** Dado un usuario registrado, Cuando ingresa una clave errónea en POST /api/v1/auth/login, Entonces el sistema retorna 401 Unauthorized y no genera token JWT.

**FICHA HU-04: Como usuario quiero registrar una nueva película o serie para proponerla al catálogo.**

- **Prioridad / RF:** Alta | RF-04, RF-05, RF-06, RNF-05
- **Actor:** Administrador / Usuario
- **Criterios de Aceptación:**
    a. Valida título, descripción, categoría, año.
    b. Rechaza títulos duplicados.
    c. Nuevas entradas se registran con `aprobado = false` (`0`).
    d. Solo Admin puede actualizar el estado de aprobación a `true` (`1`).
- **Especificación BDD / Gherkin (HU-03):**
    a. **Escenario 01 (Registro Pendiente por Usuario):** Dado un usuario regular autenticado, Cuando registra una película mediante POST /api/v1/contenidos, Entonces la entrada se crea con valor aprobado=0 y no aparece en el catálogo público.
    b. **Escenario 02 (Aprobación por Admin):** Dado un administrador autenticado, Cuando ejecuta PATCH /api/v1/contenidos/:id/aprobar, Entonces el atributo en MySQL cambia a aprobado=1 y pasa a estar visible para todos.

**FICHA HU-06: Como usuario autenticado quiero publicar una reseña y calificación en un contenido para compartir mi opinión.**

**FICHA HU-03: Como administrador quiero crear y gestionar categorías de contenido para clasificar la oferta audiovisual.**

- **Prioridad / RF:** Media | RF-07, RF-03, RNF-01
- **Actor:** Administrador
- **Criterios de Aceptación:**
    a. Solo accesible mediante Token JWT con rol ADMIN.
    b. Permite crear, listar, editar y eliminar categorías no asociadas a contenidos.
- **Especificación BDD / Gherkin:**
    a. **Escenario:** Dado un administrador autenticado, Cuando envía POST /api/v1/categorias con nombre "Cyberpunk", Entonces la categoría se registra en MySQL y retorna HTTP status 201.
- **Prioridad / RF:** Alta | RF-08, RNF-02, RNF-03
- **Actor:** Usuario Autenticado
- **Criterios de Aceptación:**
    a. Calificación en rango continuo de 1 a 10.

    b. Inserción de reseña y actualización del promedio general dentro de una transacción SQL (`BEGIN TRANSACTION` / `COMMIT`).
    c. Ejecución de `ROLLBACK` ante fallos intermedios.
- **Especificación BDD / Gherkin (HU-04):**
    a. **Escenario 01 (Publicación Transaccional Exitosa):** Dado un usuario autenticado que no ha reseñado la película X, Cuando envía POST /api/v1/reseñas con rating 9, Entonces se abre una transacción SQL, se inserta la reseña, se recalcula el rating promedio global en la tabla película, se ejecuta COMMIT y devuelve 201 Created.
    b. **Escenario 02 (Rollback por Fallo):** Si ocurre un fallo en el servidor durante el recálculo del rating promedio, Entonces la transacción ejecuta ROLLBACK y la reseña no queda guardada en la base de datos.

**FICHA HU-07: Como usuario autenticado quiero reaccionar con Like o Dislike en la reseña de otro usuario para expresar mi acuerdo o desacuerdo.**

- **Prioridad / RF:** Media | RF-09, RNF-03
- **Actor:** Usuario Autenticado
- **Criterios de Aceptación:**
    a. Restricción de autoreacción (el usuario no puede votar su propia reseña).
    b. Un solo voto por usuario con restricción `UNIQUE (user_id, review_id)`.
    c. Actualización de contadores dentro de una transacción SQL.
- **Especificación BDD / Gherkin (HU-05):**
    a. **Escenario 01 (Voto Válido):** Dado un usuario autenticado viendo la reseña de otro usuario, Cuando presiona "Like", entonces se inserta el voto y se incrementa el contador en una transacción atómica SQL.
    b. **Escenario 02 (Bloqueo de Autoreacción):** Dado un usuario autenticado intentando votar su propia reseña, Cuando ejecuta el endpoint de reacción, Entonces el servidor responde con 403 Forbidden y la restricción rechaza el evento.

**FICHA HU-08: Como usuario quiero visualizar el ranking ponderado de contenidos para descubrir el material mejor valorado.**

- **Prioridad / RF:** Alta | RF-10, RF-11, RNF-01
- **Actor:** Todos los Usuarios
- **Criterios de Aceptación:**
    a. Ejecuta el recálculo matemático de puntuación bayesiana.
    b. Ordena el catálogo de forma descendente por puntaje final.
- **Especificación BDD / Gherkin:**
    a. **Escenario:** Dado un catálogo con múltiples películas evaluadas, Cuando el cliente consulta GET /api/v1/ranking, Entonces la API devuelve el listado ordenado según la fórmula bayesiana ponderada.

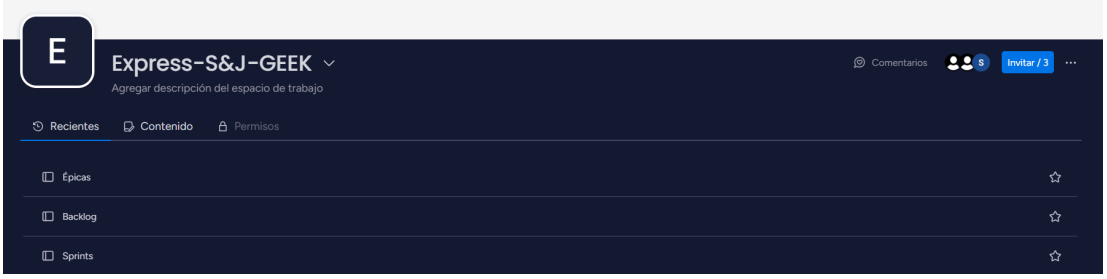
**FICHA HU-09: Como usuario quiero interactuar con la interfaz gráfica dinámica para explorar y filtrar el catálogo sin recargar la página.**

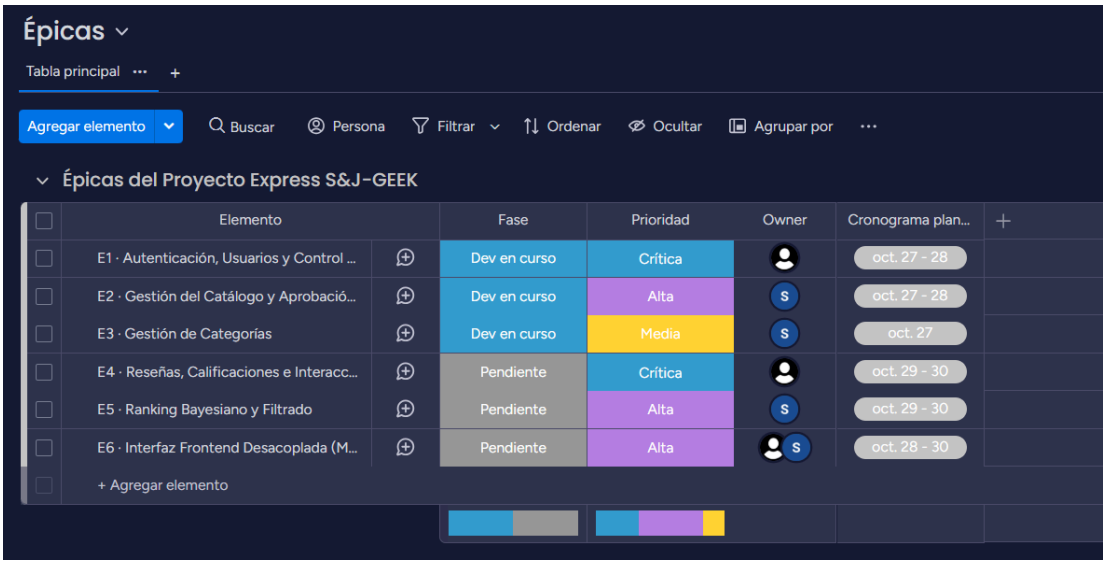
- **Prioridad / RF:** Alta | RF-11, RF-12, RNF-07, RNF-08
- **Actor:** Usuario Final / Administrador
- **Criterios de Aceptación:**
    a. Renderizado dinámico del DOM sin recargas completas (SPA-like experience con Fetch API).
    b. Diseño adaptativo (Responsive Web Design) con CSS Grid/Flexbox.
- **Especificación BDD / Gherkin:**
    a. **Escenario:** Dado un visitante en la web S&J-GEEK, Cuando selecciona el filtro "Anime", Entonces el DOM se actualiza asíncronamente mostrando únicamente contenidos de esa categoría sin refrescar la página.

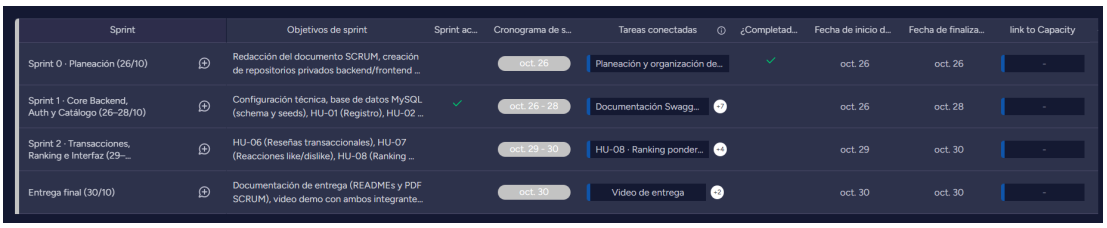
---

# 06 | Herramienta de Seguimiento y Evidencias

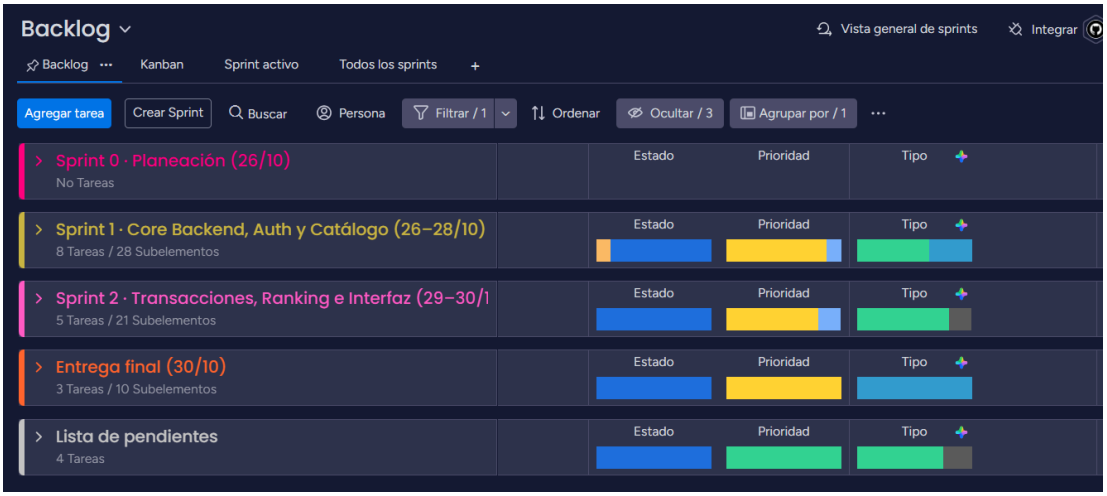
- **Herramienta Seleccionada:** Monday
- **Enlace al Tablero:** https://jakelinquino303s-team.monday.com/workspaces/18190428
- **Tipo de Acceso:** Privado con acceso a docentes
- **Flujo de Trabajo:** Establecido en el espacio de trabajo
- **Imágenes - SCRUM:**





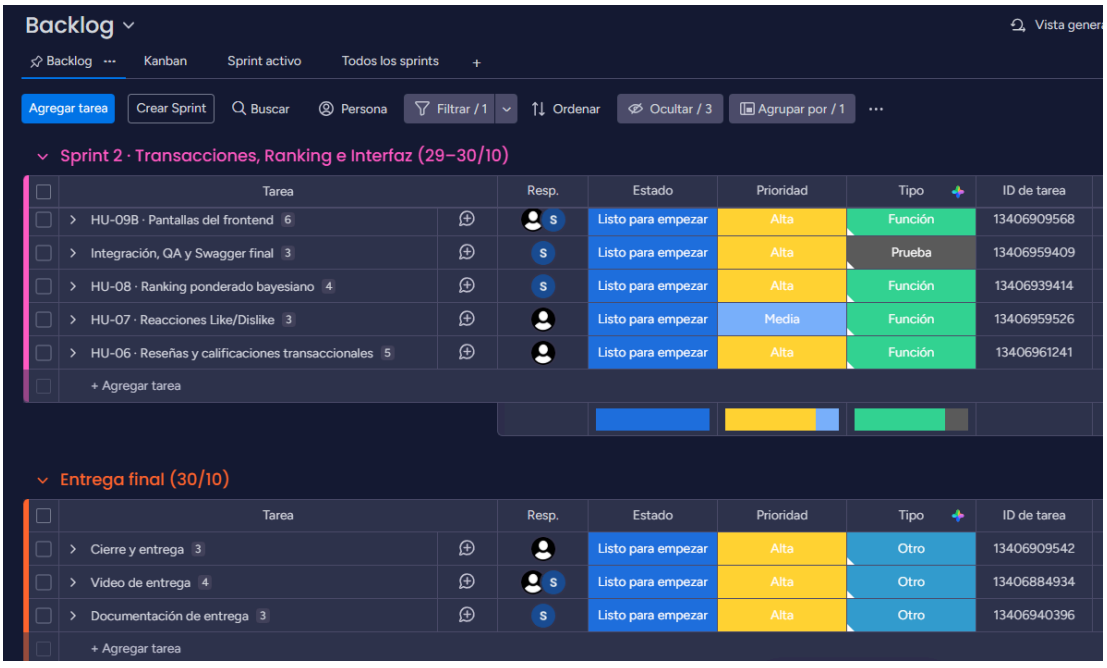
| Sprint | Objetivos de sprint | Sprint ac... | Cronograma de s... | Tareas conectadas | ¿Completad... | Fecha de inicio d... | Fecha de finaliza... | link to Capacity |
|---|---|---|---|---|---|---|---|---|
| Sprint 0 · Planeación (26/10) | Redacción del documento SCRUM, creación de repositorios privados backend/frontend ... | | oct. 26 | Planeación y organización de... | ✓ | oct. 26 | oct. 26 | - |
| Sprint 1 · Core Backend, Auth y Catálogo (26–28/10) | Configuración técnica, base de datos MySQL (schema y seeds), HU-01 (Registro), HU-02 ... | ✓ | oct. 26 - 28 | Documentación Swagg... +7 | | oct. 26 | oct. 28 | - |
| Sprint 2 · Transacciones, Ranking e Interfaz (29–... | HU-06 (Reseñas transaccionales), HU-07 (Reacciones like/dislike), HU-08 (Ranking ... | | oct. 29 - 30 | HU-08 · Ranking ponder... +4 | | oct. 29 | oct. 30 | - |
| Entrega final (30/10) | Documentación de entrega (README's y PDF SCRUM), video demo con ambos integrante... | | oct. 30 | Video de entrega +2 | | oct. 30 | oct. 30 | - |






Sprint 1 · Core Backend, Auth y Catálogo (26–28/10) — oct. 26 - 28

| Tarea | Resp. | Estado | Prioridad | Tipo | ID de tarea | SP estimado |
|---|---|---|---|---|---|---|
| Documentación Swagger del Sprint 1 2 | | Listo para empe... | Alta | Otro | 13406963107 | |
| HU-05 · Aprobación de contenidos por administrador 3 | S | Listo para empe... | Alta | Función | 13406925097 | 2 SP |
| HU-02 · Inicio de sesión y JWT 4 | | Listo para empe... | Alta | Función | 13406962366 | 3 SP |
| HU-09A · Base del frontend 3 | | Listo para empe... | Alta | Otro | 13406995689 | 2 SP |
| HU-01 · Registro de usuarios 4 | | Listo para empe... | Alta | Función | 13406961779 | 3 SP |
| Configuración técnica y base de datos 5 | S | En curso | Alta | Otro | 13406973472 | |
| HU-04 · Registro y listado de contenidos 4 | S | Listo para empe... | Alta | Función | 13406925848 | 5 SP |
| HU-03 · Gestión de categorías 3 | S | Listo para empe... | Media | Función | 13406985463 | 2 SP |
| + Agregar tarea | | | | | | |
| | | | | | | 17 SP Total |




Sprint 2 · Transacciones, Ranking e Interfaz (29–30/10)

| Tarea | Resp. | Estado | Prioridad | Tipo | ID de tarea |
|---|---|---|---|---|---|
| HU-09B · Pantallas del frontend 6 | S | Listo para empezar | Alta | Función | 13406909568 |
| Integración, QA y Swagger final 3 | S | Listo para empezar | Alta | Prueba | 13406959409 |
| HU-08 · Ranking ponderado bayesiano 4 | S | Listo para empezar | Alta | Función | 13406939414 |
| HU-07 · Reacciones Like/Dislike 3 | | Listo para empezar | Media | Función | 13406959526 |
| HU-06 · Reseñas y calificaciones transaccionales 5 | | Listo para empezar | Alta | Función | 13406961241 |
| + Agregar tarea | | | | | |

Entrega final (30/10)

| Tarea | Resp. | Estado | Prioridad | Tipo | ID de tarea |
|---|---|---|---|---|---|
| Cierre y entrega 3 | | Listo para empezar | Alta | Otro | 13406909542 |
| Video de entrega 4 | S | Listo para empezar | Alta | Otro | 13406884934 |
| Documentación de entrega 3 | S | Listo para empezar | Alta | Otro | 13406940396 |
| + Agregar tarea | | | | | |

### 6.1 Algoritmo de Ranking Ponderado (Media Bayesiana Completa)

Para evitar sesgos por películas con una sola calificación de 10/10, S&J-GEEK implementa la fórmula de Media Bayesiana Ponderada con ajuste por interacciones (Likes/Dislikes):

**Puntuación Final = (v / (v + m)) * R + (m / (v + m)) * C + Bonus_Reacciones**

- **Donde:**
  - **v:** Número total de calificaciones/reseñas que ha recibido la película específica.
  - **m:** Mínimo de votos requeridos para entrar en la ponderación oficial (constante m = 5).
  - **R:** Promedio aritmético simple de las calificaciones recibidas por la película (1 a 10).
  - **C:** Promedio global de calificaciones de todas las películas registadas en la plataforma.
  - **Bonus_Reacciones:** Factor ajustado = ((Likes - Dislikes) / (Likes + Dislikes + 1)) * 0.5.

---

## 07 | Pruebas y Cierre

- **Repositorio Backend:** Código fuente Node.js, Express, MySQL, Swagger, JWT.
- **Repositorio Frontend:** Código fuente HTML, CSS y JS puro.
- **Documentación Swagger:** `http://localhost:PORT/api-docs`

### 7.1 Matriz de Verificación de Casos de Prueba (QA Matrix)

| ID Test | Módulo Evaluado | Condición / Entrada | Resultado Esperado | Estado |
|---|---|---|---|---|
| **TC-01** | Autenticación (RF-01) | Registro con email ya registrado | Error 409 Conflict | PENDIENTE |
| **TC-02** | Seguridad RBAC (RF-03) | Usuario regular intenta aprobar película | Error 403 Forbidden | PENDIENTE |
| **TC-03** | Transacciones (RF-08/RNF-03) | Fallo simulado en update de rating | Rollback completo SQL | PENDIENTE |
| **TC-04** | Reacciones (RF-09) | Usuario vota su propia reseña | Error 400 Bad Request | PENDIENTE |
| **TC-05** | Ranking (RF-10) | Cálculo con película de 1 voto (10/10) | Ponderado bayesiano bajo | PENDIENTE |